

PetalEdge Feature Reference

The production Image Object Detection pipeline, explained in depth

SOURCE: THE APPLICATION CODE ITSELF, NOT MARKETING COPY

PetalEdge is organized as one linear pipeline — data in, a configured impulse in the middle, a deployed binary out — and every entry in its sidebar is one stage of that pipeline. This reference covers the pipeline as it exists in production today — Image Object Detection — and explains what each stage actually does, in the depth an engineer evaluating the platform would want, straight from the application's own source code.

- 01 Dashboard
- 02 Devices
- 03 Data acquisition
- 04 Data labeling
- 05 Impulse design — the pipeline concept
- 06 DSP processing — the Image block
- 07 Learning blocks — object detection architectures
- 08 Evaluation
- 09 Retrain model
- 10 Live classification
- 11 Model testing
- 12 Post-processing
- 13 Deployment (7 targets)
- 14 Jobs
- 15 Versioning

[01 · PROJECT HOME](#)

Dashboard

[/dashboard](#)

The landing page for a project, and the project switcher for everything else.

Every PetalEdge project is a self-contained workspace — its own dataset, impulses, trained models, and devices. The Dashboard is where that workspace starts: project metadata, a summary of what has been collected and trained so far, and the control for creating a new project or switching to an existing one.

The Danger Zone lives here too: performing a train/test split across the whole dataset, wiping all data in a project, or deleting the project outright. These are destructive, project-wide operations deliberately kept in one place rather than scattered across pages.

[02 · HARDWARE](#)

Devices

[/dashboard/devices](#)

Every board connected to a project, as a known, authenticated entity.

A device in PetalEdge isn't just a data source — it's a registered identity with its own authentication and connection state. When a device connects over the existing WebSocket protocol, it identifies itself, and the backend either creates a new device row or resumes an existing one for that same hardware.

What this page shows and controls:

- Identity & status. Device type, firmware version, and a live online/offline pill driven by a heartbeat — a device is considered online while it holds an active connection or has reported in recently, not from a stale last-seen timestamp.
- Device catalog. A separate, searchable catalog of 69 supported boards — not devices you own, but hardware profiles PetalEdge knows the specs and constraints of, used to resolve deployment compatibility before you build.
- Device keys. Per-device API keys, HMAC-signed, used to authenticate sample uploads and heartbeats without exposing a user's own login credentials to firmware.
- OTA update history. Once a device has a deployment on it, updates can be pushed to it later without a physical re-flash — this page shows what version is on a device and its update history.

[03 · DATASET](#)

Data acquisition

[/dashboard/data](#)

Where a dataset enters a project — from a file, or from a live device.

This is the front door for raw data. A project accepts images individually or in batch, including importing an existing folder tree with Pascal VOC-style annotations already attached — so a dataset labeled elsewhere doesn't need to be re-labeled inside PetalEdge to be usable.

[04 · DATASET](#)

Data labeling

[/dashboard/data/dataset](#)

Browse, label, and curate every sample in the dataset.

The main dataset-management surface, and the largest single page in the app. Every sample collected or uploaded lands here for browsing, filtering, and labeling before it can be used in an impulse.

- Labeling. For Object Detection, an annotation editor for drawing bounding boxes by hand, plus an AI-assisted mode that proposes boxes automatically for approval or rejection — tracked as a background job, not a blocking wait.
- Labeling queue. An unlabeled-only view with a live remaining count, previous/next navigation, and auto-advance once a sample is saved — built for working through a backlog rather than hunting for what's left.
- Bulk operations. Relabel, delete, or move samples between the train and test splits in batches, rather than one at a time.
- Synthetic data. Generate additional training images from a text prompt (via a user-supplied OpenAI API key, never stored or logged server-side), with label, count, split, size, and quality controls — useful for filling out a thin class before a training run.
- Data Explorer. A labeled 2-D scatter plot of the dataset's own feature space: your current impulse's DSP output, reduced to two dimensions by PCA, colored by class, with hover detail and click-to-inspect back to the original sample. This is where visually similar-but-mislabeled samples, or classes that overlap more than expected, actually become visible.

05 · PIPELINE

Impulse design — the pipeline concept

An impulse is the configured path your data takes from raw sample to trained model.

Every impulse has exactly three parts, always in this order: an input block (the image dimensions and resize behavior every sample is normalized to), one or more processing (DSP) blocks that turn a raw sample into a numeric feature vector, and one learning block that trains a model on those feature vectors. Processing blocks chain: the second block's input is the first block's output, all the way down the line.

A project can hold several impulses at once (visible in the sidebar's impulse switcher, with “Manage impulses” and “Create new impulse” actions), so different DSP or model configurations can be tried side by side against the same dataset without overwriting each other.

The sidebar's “Impulse design” section is generated live from whichever impulse is active: one nav item per DSP block currently in the pipeline (linking to that block's own parameters page), then one item per learning block, followed by a fixed tail — Evaluation, Retrain model, Live classification, Model testing, Post-processing, Deployment, and Jobs — that's identical regardless of which impulse you're on.

06 · PIPELINE

DSP processing — the Image block

Turning a raw uploaded image into the exact tensor the model trains on.

A processing block is a pure function: raw sample in, a numeric feature vector out. In production, every Image Object Detection impulse runs exactly one processing block — Image — which does the following:

Decodes an uploaded or captured image (JPEG, PNG, or raw uncompressed pixel bytes from camera firmware that doesn't encode), resizes it to the impulse's configured width and height under one of three modes — Fit shortest axis (scale and center-crop), Fit longest axis (scale and pad), or Squash (stretch to fit, ignoring aspect ratio) — optionally converts to grayscale, and optionally normalizes pixel values to a 0–1 range. The output is a consistent (height, width, channels) tensor no matter what size or aspect ratio the original image arrived in, which is what makes every downstream sample comparable to every other one.

07 · PIPELINE

Learning blocks

The architectures a production Image Object Detection impulse can actually train.

A learning block takes the feature vectors the Image block produced and trains a classifier or detector on them, through the existing Keras/TensorFlow training worker, on CPU or GPU queues.

Conv2D

A general-purpose 2D convolutional classifier for image-shaped input: a configurable Conv2D + batch-norm + max-pool stack (default 16 → 32 → 64 filters), global-average-pooled into dense layers. The plain-CNN alternative to MobileNet when a custom, smaller architecture is preferred over a known backbone.

MobileNet

A MobileNetV2-based image classifier — the efficient-CNN choice for whole-image classification (“what is this one picture of”) as opposed to object detection (“where are the objects in this picture”).

FOMO (marketed as NanoVision)

“Faster Objects, More Objects” — a MobileNetV2 backbone (0.35 width multiplier) with its classification head replaced by a per-pixel/per-grid-cell heatmap, so instead of drawing bounding boxes it predicts which coarse grid cell each object's center falls into. This trades precise box coordinates for a much smaller, much faster model — the object-counting/presence detector for constrained hardware. Two internal versions exist: v1 (legacy 96×96 input, stride-16 grid) and v2 (adaptive resolution, stride-8 grid, finer localization), selected via a parameter rather than a separate catalog entry.

YOLO Pro (marketed as Vision Pro)

An in-house, anchor-free, multi-scale object detector in the YOLO family, available in five size presets (nano through large) trading accuracy for speed/footprint. Requires a square-ish input whose height and width are both multiples of 32 (320×320 or 416×416 are typical), and produces real bounding boxes with class labels — the full detection task, not FOMO's presence-only heatmap.

MobileNetV2-SSD

A third detection architecture alongside FOMO and YOLO Pro — the classic Single Shot Detector head over a MobileNetV2 backbone, with its own calibration-adaptation step in the training pipeline. Gives a third accuracy/speed/footprint tradeoff point beyond the other two.

TWO CATALOG ENTRIES THAT AREN'T DIRECTLY TRAINABLE YET

“Object Detection” as a generic catalog entry has no model builder of its own — picking object detection in practice means choosing one of FOMO, YOLO Pro, or MobileNetV2-SSD explicitly, each of which is fully implemented.

“Anomaly Detection (GMM)” is listed in the learning-block catalog, but a Gaussian-mixture model needs a different training pipeline than the Keras path the other blocks use, and that pipeline isn't wired up yet — selecting it in the standard training flow is not currently a working path.

08 · MODEL QUALITY

Evaluation

</dashboard/impulse/evaluation>

How a trained model is actually scoring, and how its variants compare.

Reports the metrics that matter for the task at hand. For a classifier, that's a confusion matrix and per-class F1/precision/recall. For an object detector, background is treated specially — an image with no objects in it isn't just "wrong," it's a background false positive, tracked as its own rate (background images evaluated, and how many of those drew a spurious detection), alongside F1/precision/recall computed on the non-background classes only, which is the number that actually reflects detection quality.

Beyond raw accuracy, this page compares variants of the same trained model — different quantization or export choices — side by side on engine, inference latency, peak RAM, and flash usage, so a choice between (for example) float32 and int8 is made on real on-device numbers, not a guess. Training graphs (loss/accuracy per epoch, train vs. validation) are available from the same page for diagnosing over/underfitting.

09 · TRAINING

Retrain model

</dashboard/impulse/retrain>

Re-run training against updated data without rebuilding the impulse.

For the common case — you added more samples, or relabeled a few, and want a fresh model without re-deciding every architecture and hyperparameter — this re-runs training with the last job's configuration already carried over. It's a live view: expandable training output, real-time status (running, completed, failed, cancelled — including a genuine cancelled state that never gets silently reported back as a failure), and a cancel control for a run in progress.

10 · REAL-TIME

Live classification

</dashboard/impulse/live-classification>

Point a connected device at the model you just trained and watch it classify, live.

Runs inference against a connected camera's live feed, frame by frame, in a session you explicitly start and stop. Predictions and confidence arrive as they happen; nothing from a live session is written to the dataset unless you separately choose to record it. This is the fastest way to sanity-check a model against reality rather than a held-out test split.

11 · OFFLINE EVALUATION

Model testing

</dashboard/impulse/model-testing>

Batch-run a model over chosen samples and see exactly where it went wrong.

The offline counterpart to Live Classification: select existing samples (or upload new ones the model has never seen), run them all through the current model in one batch, and inspect the results — an accuracy summary, a full metrics table, and a per-sample inspector with thumbnails so a specific misclassification can be opened and looked at directly, not just counted. Useful for building a held-out regression set you re-run every time a model changes, independent of whatever the training split happened to be.

12 · OUTPUT TUNING

Post-processing

</dashboard/impulse/post-processing>

Tune what counts as a real detection before a raw model output becomes a final answer.

A raw model output is rarely the final answer you want — a detector fires on borderline confidence, or flickers frame to frame. This stage sits between inference and “final prediction” and is previewable directly against an uploaded project video before it's ever applied on-device.

<code>threshold</code>	Minimum confidence to count as a detection (default 0.5)
<code>tracking_enabled</code>	Track objects across frames instead of scoring each frame independently (default off)
<code>keep_grace</code>	How many frames a tracked object survives after it stops being detected, before it's dropped (default 3)
<code>max_observations</code>	Maximum simultaneous tracked objects kept for stability (default 5)
<code>class_filter</code>	Restrict tracking/output to specific classes only (default: all classes)

[13 · SHIPPING](#)

Deployment

[/dashboard/impulse/deployment](#)

Package a trained model to run on the hardware it's actually going to run on.

Deployment resolves a (model, target) pair against a compatibility service before committing to a build — estimating whether the model will fit and roughly how it will perform on that target — then produces a downloadable package. Seven targets exist in the backend today:

- Generic TFLite — a plain `.tflite` file for any runtime that can load one; the most portable option.
- Raspberry Pi — a Linux/SBC-targeted package, part of the same family as the `.pxe` runtime below.
- UNOQ — PetalEdge's own Linux/SBC runtime target, with its own on-device DSP and inference engine, not just a TFLite wrapper.
- Arduino and ESP32 — embedded microcontroller targets, generating firmware-ready code rather than a Python-runnable package.
- `cpp` — generic C++ source output for a custom embedded integration.
- `.pxe` — a packaged runtime bundle covering the Linux/SBC-class targets (TFLite, Raspberry Pi, UNOQ), which ship their own feature-extraction code and accept any DSP block those runtimes implement.

EMBEDDED MCU TARGETS SUPPORT THE IMAGE PATH

The embedded C targets (Arduino, ESP32, `cpp`) generate on-device feature code for the Image block used by every production Image Object Detection impulse — resized, normalized pixels in, the same shape the model trained on.

Once deployed, a device doesn't need a physical re-flash to get a newer model — the Devices page's OTA update flow pushes a new build to devices already running an earlier one.

[14 · WORKFLOW](#)

Jobs

[/dashboard/jobs](#)

Every background task in the project, in one place, with real status.

DSP feature generation, training runs, deployment builds, and labeling jobs all execute asynchronously on Celery queues (separate queues for CPU work, GPU training, deployment builds, DSP, and post-processing) — this page is the single place to see what's running, what finished, what failed, and what was deliberately cancelled, without hunting across the page each job type happens to live on.

[15 · SNAPSHOTS](#)

Versioning

[/dashboard/versions](#)

A full, independent copy of a project at a point in time.

This page is deliberately project-level, not impulse-level — it reads no impulse state at all, so switching the active impulse in the sidebar never reloads or refilters it. "Versioning" here means cloning: the entire project — dataset, labels, impulses, trained models, and training history — is duplicated into a new, fully independent project with its own ids throughout. It's a snapshot you can keep working from without risking the original, rather than a lightweight save-point you restore back into the same project.